

Mutable and Immutable

As noted earlier, all objects in Python have a value, data type, and identity, and of these only the value can change. If you can change the object's value, it's a *mutable* object. If you can't change its value, it's an *immutable* object. [Table 7-2](#) lists some mutable and immutable data types in Python.

[Table 7-2](#): Some of Python's Mutable and Immutable Data Types

Mutable data types	Immutable data types
List	Integer
Dictionaries	Floating-point number
Sets	Boolean
Bytearray	String
Array	Frozen set
	Bytes
	Tuple

When you overwrite a variable, it might look like you're changing the object's value, as in this interactive shell example:

```
>>> spam = 'hello'
>>> spam
'hello'
>>> spam = 'goodbye'
>>> spam
'goodbye'
```

But in this code, you haven't changed the 'hello' object's value from 'hello' to 'goodbye'. They're two separate objects. You've only switched `spam` from referring to the 'hello' object to the 'goodbye' object. You can check whether this is true by using the `id()` function to show the two objects' identities:

```
>>> spam = 'hello'
>>> id(spam)
40718944
>>> spam = 'goodbye'
>>> id(spam)
40719224
```
